

2024 年非专业级别软件能力认证第一轮

(CSP-J) 入门级 C++语言模拟试题

考生注意事项:

试题纸共有 7 页, 答题纸共有 1 页, 满分 90 分。请在答题纸上作答, 写在试题纸上的一律无效。

不得使用任何电子设备(如计算器、手机、电子词典等)或查阅任何书籍资料。

一、单项选择题(共 15 题, 每题 2 分, 共计 30 分; 每题有且仅有一个正确选项)

1. 某种计算机的内存容量是 640K, 这里的 640K 容量是指()个字节。

- A. 640 B. 640x1000 C. 640 x 1024 D. 640x1024x1024

2. 一个完整的计算机系统应包括()。

- A. 系统硬件和系统软件 B. 硬件系统和软件系统
C. 主机和外部设备 D. 主机、键盘、显示器和辅助存储器

3. x 补码=10011000, 其原码为()。

- A. 011001111 B. 11101000 C. 11100110 D. 01100101

4. 运行以下代码片段的行为是()。

```
int x = 101;
int y = 201;
int *p = &x;
int *q = &y;
p = q;
```

- A. 将 x 的值赋为 201
B. 将 y 的值赋为 101
C. 将 q 指向 x 的地址
D. 将 p 指向 y 的地址

5. 以下关于排序算法的描述正确的是()。

- A. 选择排序、计数排序是稳定的排序
B. 冒泡排序、堆排序是稳定的排序
C. 冒泡排序的时间复杂度为 $O(n^2)$, 快速排序的时间复杂度为 $O(n\log n)$
D. 桶排序和归并排序是稳定的排序

6. 对于为冒泡排序来说, 最坏的情况下需要比较几次, 最好的情况下需要比较几次()。

- A. $n^2, n-1$ B. $\frac{n(n-1)}{2}, n-1$ C. $\frac{n(n-1)}{2}, n$ D. n^2, n

7. A, B, C, D, E 五人并列站成一排, 如果 A, B 必须相邻且 B 在 A 的右边, 那么不同的排法有几种 ()。

- A. 60 种 B. 48 种 C. 36 种 D. 24 种

8. 2024 个结点的四叉树有几层高 ()

- A. 5 B. 6 C. 7 D. 8

9. 与二进制数 11100.11 等值的八进制数为 ()

- A. 32.6 B. 34.6 C. 34.3 D. 32.3

10. 一个字符串中任意个连续的字符组成的子序列称为改字符串的子串, 则字符串 abcdab 有几个内容互不相同的子串 ()。

- A. 14 B. 16 C. 18 D. 20

11. 已知一棵完全二叉树的第 6 层 (设根为第 1 层) 有 8 个叶结点, 则该完全二叉树的结点个数最多是 ()。

- A. 39
B. 52
C. 111
D. 119

12. 对 n ($n \geq 2$) 个权值均不相同的字符构造成哈夫曼树。下列关于该哈夫曼树的叙述中, **错误**的是 ()。

- A. 该树一定是一棵完全二叉树。
B. 树中一定没有度为 1 的结点。
C. 树中两个权值最小的结点一定是兄弟结点。
D. 树中任一非叶结点的权值一定不小于下一层任一结点的权值。

13. 若无向图 $G=(V, E)$ 中含有 7 个顶点, 要保证图 G 在任何情况下都是连通的, 则需要的边数最少是 ()。

- A. 6 B. 15 C. 16 D. 21

14. A、B、C 三个社区需要建设若干个 5G 基站, 三个社区可供选择的建设基站地点分别有 2 个、4 个、5 个, 现从 A、B、C 三个社区分别选取 1、2、3 个地点随机分配给甲、乙、丙三个施工队进行建设, 要求每个施工队只能承接一个社区, 则承建方式有 ()。

- A. 720 种 B. 480 种 C. 360 种 D. 120 种

15. 以下不属于编程语言的是 ()。

- A. VB B. PHP C. Android D. Pascal

二、阅读程序（程序输入不超过数组或字符串定义范围；判断题正确填√，错误填×；出特殊说明外，判断题2分，选择题3分，合计30分）

(1)

```
1  #include <iostream>
2  using namespace std;
3  const int maxn = 1000;
4  long long a[maxn];
5
6  long long Fibonacci(int n) {
7      if (n == 1 || n == 2)
8          return 1;
9      if (n == 3)
10         return 2;
11     if (n == 4)
12         return 3;
13     if (a[n])
14         return a[n];
15     return a[n] = Fibonacci(n - 1) + Fibonacci(n - 2);
16 }
17
18 int main() {
19     int n;
20     cin >> n;
21     int ans = Fibonacci(n);
22     cout << ans;
23     return 0;
24 }
```

判断题

16. 去掉代码的第13,14行对输出结果没有影响（ ）
17. 把第15行代码改为“return a[n] = Fibonacci(n - 2) + 2 * Fibonacci(n - 3) + Fibonacci(n - 4);”对输出结果没有影响（ ）
18. 把第22代码改为“cout << a[n];”对输出结果没有影响（ ）

单选题

19. $n = 10$ 时，输出的结果是（ ）
- A. 38 B. 55 C. 72 D. 89
20. $n = 24$ 时，输出的结果的个位数字是（ ）
- A. 1 B. 2 C. 4 D. 8
21. 程序的时间复杂度是（ ）
- A. $O(n)$ B. $O(n\log n)$ C. $O(n^2)$ D. $O(2^n)$

```

(2)
01 #include<iostream>
02 using namespace std;
03 int d[1000001];
04 long long n, m;
05 long long getSum(int high) {
06     long long alln = 0;
07     for (int i = 1; i <= n; i++) {
08         if (high < d[i]) alln += d[i] - high;
09     }
10     return alln;
11 }
12 int search(long long x, int left, int right) {
13     if (left == right) return left;
14     int mid = rand() % (right - left) + left;
15     if (getSum(mid) >= x)
16         return search(x, mid + 1, right);
17     else
18         return search(x, left, mid);
19 }
20 int main() {
21     srand(time(0));
22     cin >> n >> m;
23     for (int i = 1; i <= n; i++) cin >> d[i];
24     cout << search(m, 0, INT32_MAX) - 1;
25     return 0;
26 }

```

假设输入的所有数字是均不超过 10^6 的正整数，完成下面的判断题和单选题：
判断题

22. 把代码的 21 行去掉，对运行的结果没有影响 ()
23. 把代码的 24 行的“INT32_MAX”改为 1000000, 对运行结果没有影响 ()
24. 把代码中所有的 long long 改为 int, 对运行的结果没有影响 ()

单选题

25. 当输入为“3 6 7 7 7”时，输出为 ()
- A. 2 B. 3 C. 4 D. 5
26. 当输入为“1 5 3”时，输出为 ()
- A. 0 B. -1 C. -2 D. -3
27. 当输入为“5 20 4 42 40 26 46”时，输出为 ()

- A. 8 B. 24 C. 32 D. 36

三、完善程序（单选题，每小题 3 分、合计 30 分）

(1) (最大素因子) 现在给出 N 个序列号，每个序列号的范围在 1~20000 之间，请编程确定谁有最大的素因子。如果有多个这样的序列号，则输出输入数据中较早输入的数。如果没有素因子，则输出 0。

```
1  #include <iostream>
2  #include <cmath>
3  using namespace std;
4  int prime(int n) {
5      for (int i = 2; i <= sqrt(n); i++) {
6          if (①) {
7              ②;
8          }
9      }
10     return 1;
11 }
12 int pd(int n) {
13     for (int i = n; i > 1; i--) {
14         if (③) {
15             return i;
16         }
17     }
18     return -1;
19 }
20 int main() {
21     int n, x, max = 0, maxi = 0;
22     cin >> n;
23     for (int i = 0; i < n; i++) {
24         cin >> x;
25         if (④) {
26             max = pd(x);
27             ⑤;
28         }
29     }
30     cout << maxi;
31     return 0;
32 }
```

28. ①处应填()

- A. $n \% i == 0$ B. $n / i == 0$ C. $n / i == 1$ D. $i * i == n$

29. ②处应填()

- A. break B. continue

C. return 1

D. return 0

30. ③处应填()

A. `n % i == 1 && prime(n/i)`

B. `n % i == 0 && prime(i)`

C. `n % i == 1 && pd(n/i)`

D. `n % i == 0 && pd(i)`

31. ④处应填()。

A. `pd(x/i) >= max`

B. `prime(x) > max`

C. `pd(x) > max`

D. `prime(x/i) >= max`

32. ⑤处应填()。

A. `maxi = x`

B. `maxi = max`

C. `maxi = prime(x)`

D. `maxi = x/i`

(2) (轮流取数) 给定一组数字, 玩家 A 和 B 轮流取数字, A 先开始。每次只能取最后一个数字或者第一个数字。如果最终玩家 A 所取得的数字之和大于等于玩家 B, 则认为玩家 A 胜。问在 A 和 B 都在使用最优策略的情况下 A 是否必胜。试补全动态规划算法。

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4  const int maxn = 2000;
5  int c[maxn];
6  int main() {
7      int n;
8      cin >> n;
9      vector<vector<int>> dp(n + 1, vector<int>(n + 1));
10     for (int i = 1; i <= n; i++) {
11         cin >> c[i];
12         ①;
13     }
14     ②;
15     for (int left = 1; ③; left++) {
16         dp[cnt][left] = ④;
17     }
18     if (⑤) {
19         cout << "true";
20     } else {
21         cout << "false";
22     }
23     return 0;
24 }
```

33. ①处应填()

- A. `dp[0][i] = c[i]` B. `dp[i][0] = c[i]`
C. `dp[1][i] = c[i]` D. `dp[i][1] = c[i]`

34. ②处应填()

- A. `for (int cnt = 0; cnt < n; cnt++)`
B. `for (int cnt = 1; cnt < n; cnt++)`
C. `for (int cnt = 1; cnt <= n + 1; cnt++)`
D. `for (int cnt = 2; cnt <= n; cnt++)`

35. ③处应填()

- A. `left <= n` B. `left + cnt + 1 <= n`
C. `left + cnt <= n` D. `left + cnt - 1 <= n`

36. ④处应填()

- A. `max(c[left] - dp[cnt - 1][left + 1], c[left + cnt - 1] - dp[cnt - 1][left])`
B. `min(c[left] - dp[cnt - 1][left + 1], c[left + cnt - 1] - dp[cnt - 1][left])`
C. `max(c[left] + dp[cnt - 1][left + 1], c[left + cnt - 1] + dp[cnt - 1][left])`
D. `min(c[left] + dp[cnt - 1][left + 1], c[left + cnt - 1] + dp[cnt - 1][left])`

37. ⑤处应填()

- A. `dp[cnt][1] != 0` B. `dp[cnt][1] <= 0`
C. `dp[cnt][1] >= 0` D. `dp[cnt][1] % 2 != 0`